IFT339 - Exercices sur les tables de hachage et les graphes

Note: dans ces exercices, sauf indication contraire on suppose qu'une table de hachage peut faire insertion/recherche/suppression en temps $O(1)$, en ignorant l'aspect amorti et avec forte probabilité.

Exercice 1.
Soit T une table de hachage de capacité m. On a vu dans l'adressage ouvert que si on veut ajouter une clé c à une position $p = h(c)$, où ici h est notre fonction de hachage, on doit parcourir les alvéoles p, $p+1$, $p+2, \ldots$ jusqu'à ce qu'on trouve une alvéole vide. Un effet indésirable de cette idée est que les valeurs ont tendance à être regroupées en blocs alors qu'on voudrait plutôt disperser au maximum.

Une idée est de changer le pas du parcours. C'est-à-dire, on définit un k et si on veut insérer c à la position $p = h(c)$, on parcourt les alvéoles dans l'ordre p, $p+k$, $p+2k$, $p+3k, \ldots$ jusqu'à ce qu'on trouve une cellule vide.

a Insérez dans une table de hachage de taille 12 (donc avec des positions de 0 à 11) avec un pas de $k = 3$ les valeurs suivantes : 3, 18, 24, 42, 45. Prenez comme fonction de hachage $h(i) = i\%12$. Donnez l'état de la table après la dernière insertion. Quel est le problème? Notez que seules deux des valeurs insérées ont la même valeur après application de h.

b Nous avons le théorème suivant, qui nous provient de la théorie des groupes (une conséquence du théorème de Lagrange):

Théorème : Soit m un nombre premier. Alors pour tout entiers $p < m$ et $k < m$, les valeurs
p,
$(p+k)\%m$,
$(p+2k)\%m$,
$(p+3k)\%m$,
$\ldots$,
$(p+(m-1)k)\%m)$
sont toutes différentes.

Utilisez le théorème pour justifier qu'il est préférable d'utiliser un nombre premier pour la capacité m de table de hachage en adressage ouvert.

Solution

Dans la première partie, le problème est qu'à seulement la 5ème insertion, on boucle à l'infinie pour trouver un endroit où insérer même si la table est pleine au 1/3.

Dans la deuxième partie, le théorème garantit que si m est premier, alors en essayant successivement d'insérer à l'alvéole p, $p+k$, et ainsi de suite, on devra parcourir chaque alvéole avant de revisiter la même deux fois. Donc, la seule façon de manquer d'espace est si la table est pleine.

Fin de la solution

Exercice 2.
On dit souvent que la complexité des opérations d'une table de hachage est $O(1)$, mais ceci suppose que la fonction de hachage est calculable en temps $O(1)$. Ceci n'est pas toujours vrai.

Supposons que vous stockez des chaînes de caractères de longueur au plus m dans une table de hachage en adressage chaîné. Donnez les complexités pour l'insertion/recherche/suppression, dans le pire cas et dans le meilleur cas (qui est lorsque chaque alvéole a un nombre $O(\alpha)$ d'éléments).

Solution

On remarque que, tel que mentionné en classe, le hachage d'une chaîne de longueur m dépend de chaque caractère de la chaîne. Il faut au moins un temps $O(m)$ pour appliquer la fonction de hachage.

Pour insérer une chaîne S, dans le pire des cas chacun des n éléments est dans la même alvéole. On doit comparer S à chaque chaîne de l'alvéole, et chaque comparaison prend un temps $O(m)$. L'ajout de S dans l'alvéole demande de faire un push_back en temps $O(1)$ (ou techniquement, s'Il faut copier S c'est $O(m)$). L'insertion demande donc un temps total de $O(m+n\cdot$

$m + 1) = O(nm)$. La recherche et la suppression demandent aussi un temps $O(nm)$ pour les mêmes raisons.

Dans le meilleur cas, l'insertion demande de faire le hachage en temps $O(m)$ et de comparer S à $O(1)$ chaînes dans son alvéoles, en temps $O(m)$ par chaîne. Le temps est donc proportionnel à une constante, fois le temps de comparaison, donc $O(m)$.

Fin de la solution

Exercice 3.
Dans le célèbre problème de la double-somme (two-sum en anglais), on a en entrée un vecteur d'entier vec et un entier t. Il faut déterminer s'il existe deux indices i et j tels que vec[i] + vec[j] = t. Donnez un code en temps $O(n)$ pour ce problème (avec n la taille de vec).

Solution

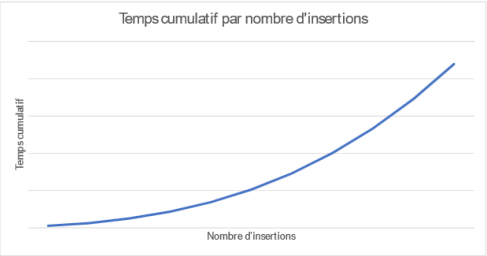
Si vous essayez toutes les paires de i, j possibles, ceci prendra un temps $O(n^2)$. On peut utiliser une table de hachage pour stocker les valeurs rencontrées, et quand on arrive à $vec[i]$ on regarde si $t - vec[i]$ a été vu avant.

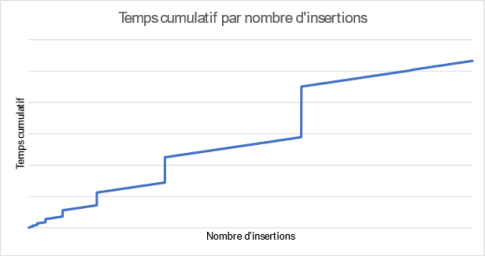
```
//clé = entier, val = position vue
unordered_set<int, int> prevs;
for (int i = 0; i < vec.size(); ++i){
    if (prev.find( t - vec[i] ) != prev.end()){
        return make_pair(prev[t - vec[i]], i);
    }
    prev[vec[i]] = i;
}
return make_pair(-1, -1);    //rien trouvé
```

Ceci prend un temps $O(n)$ si on suppose que la recherche et l'insertion dans $prev$ se fait en temps $O(1)$.

Fin de la solution

Exercice 4.
Considérez les graphiques suivants, qui montrent le temps cumulatif pour effectuer n insertions dans un arbre binaire de recherche balancé, et dans une table de hachage. Dites lequel des graphiques correspond à quelle structure de données.





Solution

À gauche, c'est un ABR balancé. Le temps cumulatif est la somme des $\log(1) + \log(2) + \ldots + \log(n)$, alors chaque insertion prend un peu plus de temps que la précédente.

À droite, c'est la table de hachage — les montées soudaines correspondent à la recréation de la table pour doubler la capacité lorsque le facteur de charge est devenu trop grand.

Fin de la solution

Exercice 5.
Vous avez une liste chaînée programmée par un stagiaire et vous suspectez que ses pointeurs peuvent être mal organisés. Il se peut qu'une liste chaîne ait un cycle, qui a la forme d'une séquence de noeuds $v1 \to v2 \to \ldots \to vk \to v1$. La notation $vi \to vj$ veut dire que $vi.next = vj$.

Montrez comment on peut détecter si une liste chaînée contient un cycle ou non en temps $O(n)$. Vous pouvez utiliser la classe suivante.

```
template <typename TYPE>
class list{
    struct Noeud{
        TYPE val;
        Noeud* next;
    }
    Noeud* debut;

    bool contient_cycle();
}
```

Solution

On stocker les noeuds rencontrés dans une liste chaînée, et si on trouve un noeud avec $vi.next$ dans ces noeuds, on a un cycle. Si on suppose un temps $O(1)$ sur les tables de hacahge, ceci prend un temps $O(n)$.

```
bool contient_cycle(){
    Noeud* v = debut;
    unordered_set<Noeud*> prevs;
    while (v){
        if (prevs.find(v->next) != prev.end())
            return false;
        prevs.insert(v);
        v = v->next;
    }
    return true;
}
```

Fin de la solution

Exercice 6.
En bioinformatique. les séquences d'ADN sont représentées par des string sur alphabet $\{A, C, G, T\}$. Étant donné une séquence S, un k-mer est une sous-chaîne de S de longueur k (donc un segment consécutif de k caractères). Par exemple, si $S = ACTACTG$ et $k = 3$, l'ensemble des k-mers est $\{ACT, CTA, TAC, CTG\}$ (notez que l'on n'a pas listé ACT deux fois car on parle d'un ensemble ici).

a On veut stocker les k-mers d'une séquence S dans une table de hachage. Donnez d'abord une bonne fonction de hachage pour accomplir cette tâche. Cette fonction devrait être indépendante de k et de la longueur de S, et bien fonctionner peut importe la capacité de la table.

Solution

Il n'est pas trivial de trouver une bonne fonction de hachage, et je n'ai pas de réponse exacte. Si on fait juste prendre la somme des codes ASCII, par exemple $hash(\text{“}ACAG\text{”}) = 27 + 29 + 27 + 34$, alors nos adresses seront toutes des entiers regroupés autour des mêmes valeurs, augmentant les chances de collision. De plus, plusieurs chaînes donnent la même somme, par exemple on aurait la même chose avec “$AACG$”.

Il faut donc que deux chaînes dont l'une est une permutation d'une autre donnent une adresse différente. On peut donner un poids différent à chaque position, par exemple en donnant un exposant différent à chaque caractère selon sa position:

$$hash(S) = ascii(S[0])^1 + ascii(S[1])^2 + \ldots + ascii(S[k])^k.$$

Ce n'est toujours pas parfait, car les valeurs d'adressess seront quand même regroupées autour de valeurs similaires. On peut utiliser le fait que C++ implémente déjà de bonnes fonctions de hachage pour les entiers. Si on hache chaque terme de la somme, ça devrait faire l'affaire.

$$hash(S) = hash(ascii(S[0])^1) + hash(ascii(S[1])^2) + \ldots + hash(ascii(S[k])^k).$$

Reste à l'essayer expérimentalement, chose que je n'ai pas faite.

Fin de la solution

b Soit deux séquences S_1 et S_2. La distance de k-mers entre S_1 et S_2 consiste en le nombre de k-mers présents dans S_1 mais pas dans S_2, plus le nombre de k-mers présents dans S_2 mais pas dans S_1. Dites comment on peut calculer cette distance efficacement, puis donnez la complexité en fonction de la taille de S_1 et S_2, et de k.

Exercice 7.
Une façon alternative de stocker un graphe est d'entreposer toutes les arêtes dans une table de hachage. Donc on ne stocke jamais les sommets individuels – que les arêtes. On suppose que l'on a implémenté une fonction de hachage appropriée pour la structure `pair<int, int>`. Chaque paire représente une arête et on ne fait que stocker chaque arête dans un `unordered_set< pair <int, int> >`. Donnez la complexité des opérations sur les graphes.

- ajouter un sommet
- supprimer un sommet
- ajouter une arête
- supprimer une arête
- obtenir le nombre de voisins d'un sommet
- déterminer si une arête est présente

Solution

On suppose que la table de hachage en question s'appelle `tab`. On dénote par m le nombre d'arêtes, et on suppose que les sommets sont $\{0, 1, \ldots, n-1\}$.

- Ajouter un sommet ne demande pas de temps avec cette implémentation: tel que spécifié dans la question, on ne stocke pas les sommets individuels.

- Pour supprimer un sommet i, il faut supprimer toutes les paires de tab qui contiennent i.

  Pour ce faire, on peut faire une itération sur la table, style:

```
auto it = tab.begin();
while (it != tab.end()){
    if ((*it).first == i || (*it).second == i)
        it = tab.erase(it);
    else
        ++it;
}
```

  ce qui prend en temps $O(cap)$, où cap est la capacité de la table, ce qui devrait en fait être $O(m)$ si α est maintenu entre $1/4$ et $1/2$.

  On peut aussi atteindre $O(n)$, en bouclant sur toutes les arêtes $\{i, j\}$ possibles et en supprimant celles qui sont présentes.

```
for (int j = 0; j < n; ++j){
    tab.erase( make_pair(i, j) );
    tab.erase( make_pair(j, i) );
}
```

  Ceci est $O(n)$ en supposant un temps constant pour supprimer.

- Ajouter une arête $\{i, j\}$ se fait en temps $O(1)$ en appelant tab.insert.

- Supprimer une arête $\{i, j\}$ se fait en temps $O(1)$ en appelant tab.erase.

- Pour obtenir le degré (nombre de voisins) d'un sommet i, je ne vois pas de meilleure façon que de regarder si $\{i, j\}$ est une arête pour chaque j possible.

```
int deg = 0;
for (int j = 0; j < n; ++j){
    if (tab.find( make_pair(i, j)) != tab.end() ){
        ++deg;
    }
```

```
}
```

ce qui prend un temps $O(n)$. Si on voulait énumérer les voisins, la complexité serait la même.

- Pour déterminer si une arête $\{i, j\}$, on appelle find qui prend un temps $O(1)$.

Fin de la solution

Exercice 8.
Considérez un graphe représenté par une liste d'adjacence, comme dans la classe ci-bas.

a Étant donné un indice i, donnez le code d'une fonction qui retourne l'ensemble de tous les sommets que i peut atteindre. On dit que i peut atteindre j s'il existe un chemin de i à j, c'est-à-dire, une liste de sommets $i - a_1 - a_2 - \ldots - a_k - j$ telle que tous les sommets qui se suivent dans la séquence partagent une arête.

Ne visez pas une complexité optimale, mais évitez un algorithme en temps exponentiel ou pire.

b Étant donné deux indices de sommets i et j, donnez le code d'une fonction qui d retourne un *chemin* de i à j, s'il en existe un.

```
class graphe{
    //adj[i] = voisins de i
    vector<int, vector<int> > adj;

    vector<int>
}
```

Solution

Notez que ce type de question ne se retrouverait pas à l'examen!

Pour le a), on énumère les voisins de i, puis les voisins des voisins, et ainsi de suite. Une fois qu'on a énuméré les voisins d'un sommet j, il faut s'assurer de ne pas ré-énumérer ses voisins encore une fois, sinon le code pourrait boucler à l'infini. On va donc maintenir un ensemble visited qui se rappelle qui a été traité ou pas. De plus, visited contiendra tous les sommets accessibles de i à la fin de l'exécution.

Chaque itération construit la liste des sommets à visiter pour la prochaine itération.

```
//je suppose que j'ai un objet g de type graphe
unoredered_set<int> visited;

unordered_set<int> todo;
todo.insert(i);

while (!todo.empty()){
    unordered_set<int> next_todo;

    for (auto it = todo.begin(); it != todo.end(); ++it){
        int j = (*it);
        visited.insert(j);  //se rappeler que j est traité
        for (int voisin : g.adj[j]){
            //trouver les voisins de j non-visités
            //et les lister pour la prochaine itération
            if (visited.find(voisin) == visited.end())
                next_todo.insert(voisin);
        }
    }
    todo = next_todo;   //prochaine itération
}
```

Je ne demandais pas de donner la complexité, mais on peut argumenter que ce code prend un temps $O(n + m)$ car il visite chaque sommet, et énumère chaque arête au plus deux fois.

Pour le b), il y a plusieurs façons de faire, je propose ici un parcours en profondeur qui construit récursivement un chemin jusqu'à ce que j soit trouvé. Ce n'est **pas le code le plus efficace** puisque les chemin partiels sont copiés à chaque appel récursif, et qu'on fait un find dans un vecteur, mais je le présente pour des fins pédagogiques. En exercice supplémentaire, trouvez comment accélérer ce code.

En exercice encore plus supplémentaire, trouvez un chemin de longueur minimum. Une bonne solution consiste à modifier l'algorithme en a).

```
//retourne un vecteur vide si aucun chemin de i à j étend
   cur_chemin, ou un chemin s'il en existe un
//L'appel initial devrait etre avec cur_chemin = [i]
vector<int> get_chemin(int i, int j, vector<int> cur_chemin){
    if (cur_chemin.back() == j)
        return cur_chemin;
    for (int k : g.adj[cur_chemin.back()]){
        if (cur_chemin.find(k) != cur_chemin.end()){
            vector<int> tmpch = cur_chemin;
            tmpch.push_back(k);
            vector<int> retch = get_chemin(i, j, tmpch);
            if (!retch.empty())
                return retch;
        }
    }
    vector<int> retch;
    return retch;
}
```

Fin de la solution